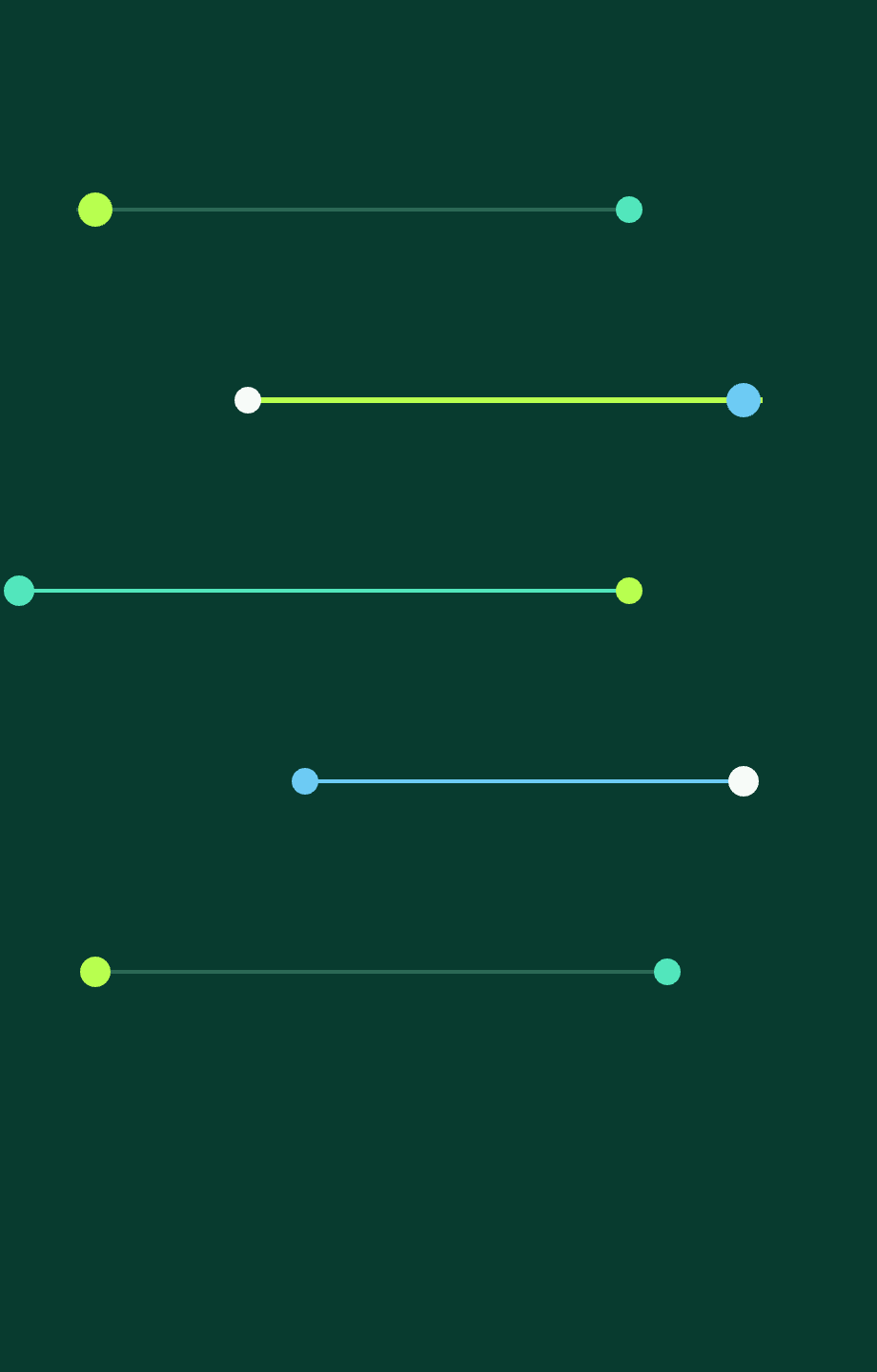


PROJECT ATLAS

A local-first AI desktop operating system for agency work

Working codename. Brand is not final.



One desktop operating layer for agency work

Atlas brings operational context, AI assistance and execution workflows into a secure local-first application.

Atlas is being built as a control plane for agency operators and teams managing several clients or businesses. The product aims to reduce context switching while keeping sensitive data and AI access under explicit application control.

Foundation is real

Tasks 001-006 are approved and committed. They cover the desktop shell, runtime services, database foundation and AI provider architecture.

First AI workflow exists

Task 007 adds Atlas AI with Ollama status, settings, persistence and single-turn generation. Final closure remains pending.

The product is still early

The broader agency operating system remains a roadmap. Future modules in this document describe intent rather than shipped capability.

Agency work fragments across records, tools and AI chats

CURRENT FRICTION

Context lives in separate systems

Client data, projects, campaign plans, analytics and delivery history rarely share a common operating model.

AI starts without trusted context

Teams repeatedly paste background into generic chat tools and manually move outputs back into operational systems.

Tool sprawl adds coordination cost

More software can create more handoffs, duplicate status tracking and inconsistent ownership.

ATLAS RESPONSE

A shared desktop workspace

Operational modules use the same local data foundation, background services and controlled AI layer.

Structured context

Clients, projects, businesses and campaigns can provide reusable context to workflows.

Governed AI

Typed requests, provider adapters and validation keep AI access behind one boundary.

Integrated execution

Automation and integrations can connect planning to delivery without rebuilding every external service.

Product thesis: combine durable operating context with AI that can act through explicit, auditable workflows.

A modular operating system with four product principles

01 Local by default

Core records, settings and supported AI workflows remain usable on the desktop without making cloud services mandatory.

02 One controlled AI boundary

Application code talks to the AI Engine. Provider-specific behavior stays behind adapters and validated contracts.

03 Shared operational context

Modules can build on the same client, project and business records instead of creating isolated data silos.

04 Explicit automation

Jobs, integrations and future actions should expose status, ownership and failure states to the operator.

Tasks 001-006 are closed. Task 007 awaits final closure

The status below reflects only the supplied project record.

001	Repository foundation	APPROVED / COMMITTED
002	Runtime services	APPROVED / COMMITTED
003	Database foundation	APPROVED / COMMITTED
004	First desktop application	APPROVED / COMMITTED
005	AI Engine core	APPROVED / COMMITTED
006	Ollama provider	APPROVED / COMMITTED
007	First Atlas AI vertical slice	IMPLEMENTED / CLOSURE PENDING

The foundation now supports one real local AI workflow

Counts summarize the stated scope. They do not indicate overall product completion.

7

scoped milestones

Tasks 001-007 in the supplied project record

6

closed milestones

Approved and committed before the current review

1

AI vertical slice

Atlas AI with Ollama settings, status and generation

3

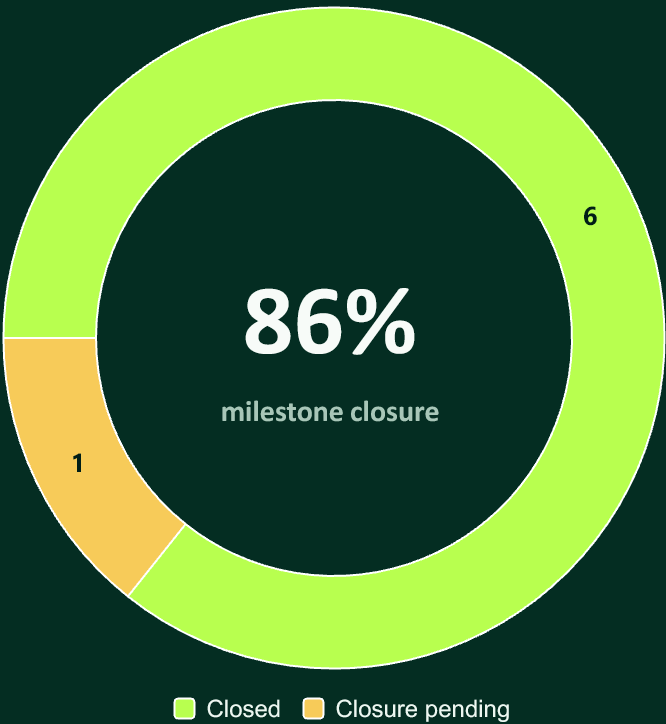
provider paths

Fake provider and Ollama exist. Gemini remains future intent

Provider count includes Gemini as a planned path, not an implemented adapter.

Six of seven scoped milestones have formal closure

Task 007 is implemented and still needs final acceptance.



Closed scope

Repository foundation, runtime services, database foundation, desktop shell, AI Engine core and Ollama provider.

Open closure item

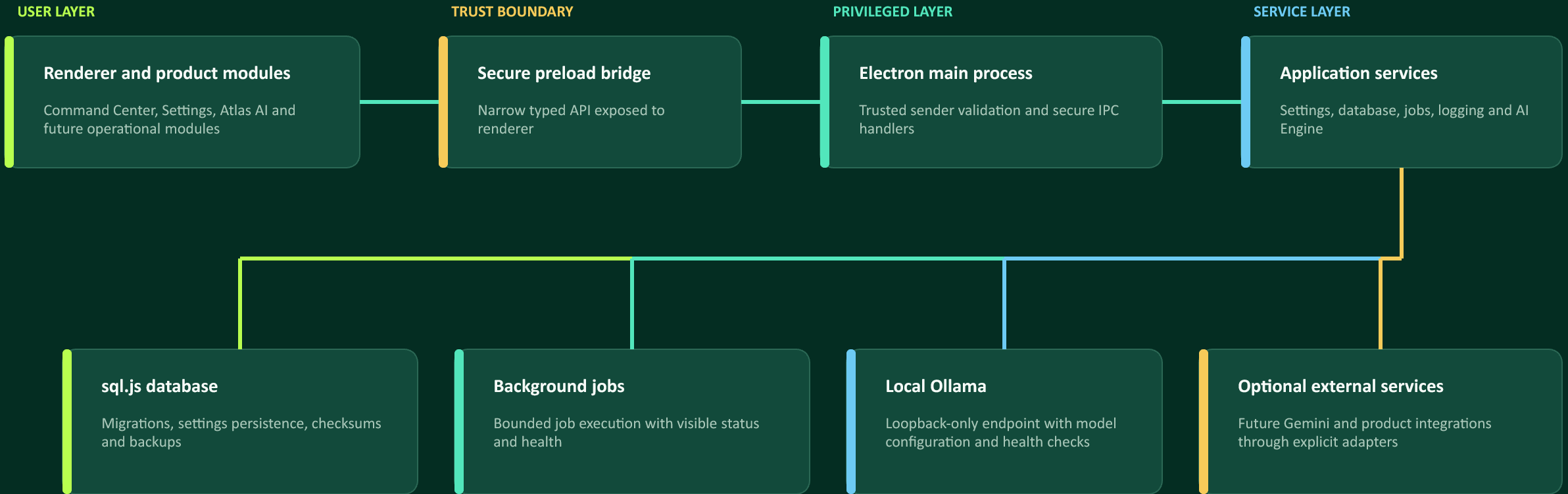
Complete Task 007 acceptance, verify the full user flow and record the final milestone decision.

86% equals 6 closed milestones divided by 7 scoped milestones. Internal milestone estimate only. It is not measured product completion.

Current capability and future product scope

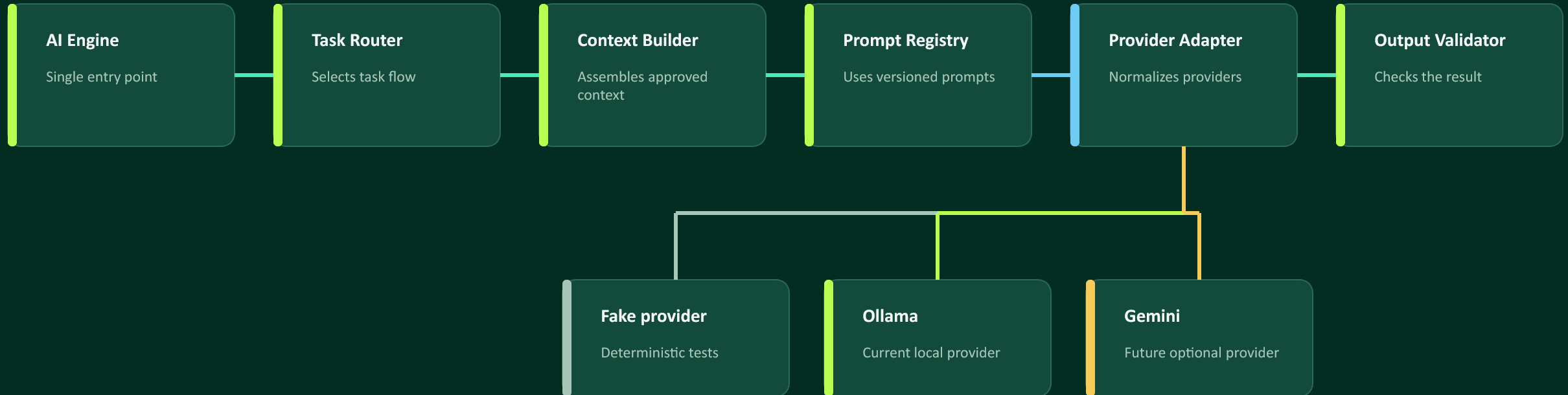
CURRENT	NEXT	FUTURE
01Secure Electron desktop shell	01Close Task 007 acceptance	01Automation and integrations
02Settings, health and background jobs	02Client and project records	02Businesses and discovery
03sql.js persistence and migrations	03Reusable operational context	03Deployments and website generation
04AI Engine with typed contracts	04Prompt and workflow expansion	04Optional Gemini provider
05Ollama status, settings and single-turn generation	05Early analytics and campaign workflows	05Plugin ecosystem and broader analytics

Atlas keeps UI, privileged services and providers in separate layers



Normal renderer code never talks directly to the database, Ollama or external providers.

One governed path from user intent to validated output



Provider-specific details stay behind adapters. Normal application code receives typed responses and normalized errors.

Defense in depth around privileged desktop capabilities

01 Renderer containment

CSP, navigation restrictions and permission controls reduce the renderer attack surface.

02 Narrow preload bridge

The renderer receives an explicit API instead of direct access to Node.js or provider internals.

03 Validated IPC

Trusted sender checks and secure handler helpers protect privileged operations.

04 Local data controls

sql.js persistence, migration checksums, backup tooling and startup health support integrity.

05 AI boundary

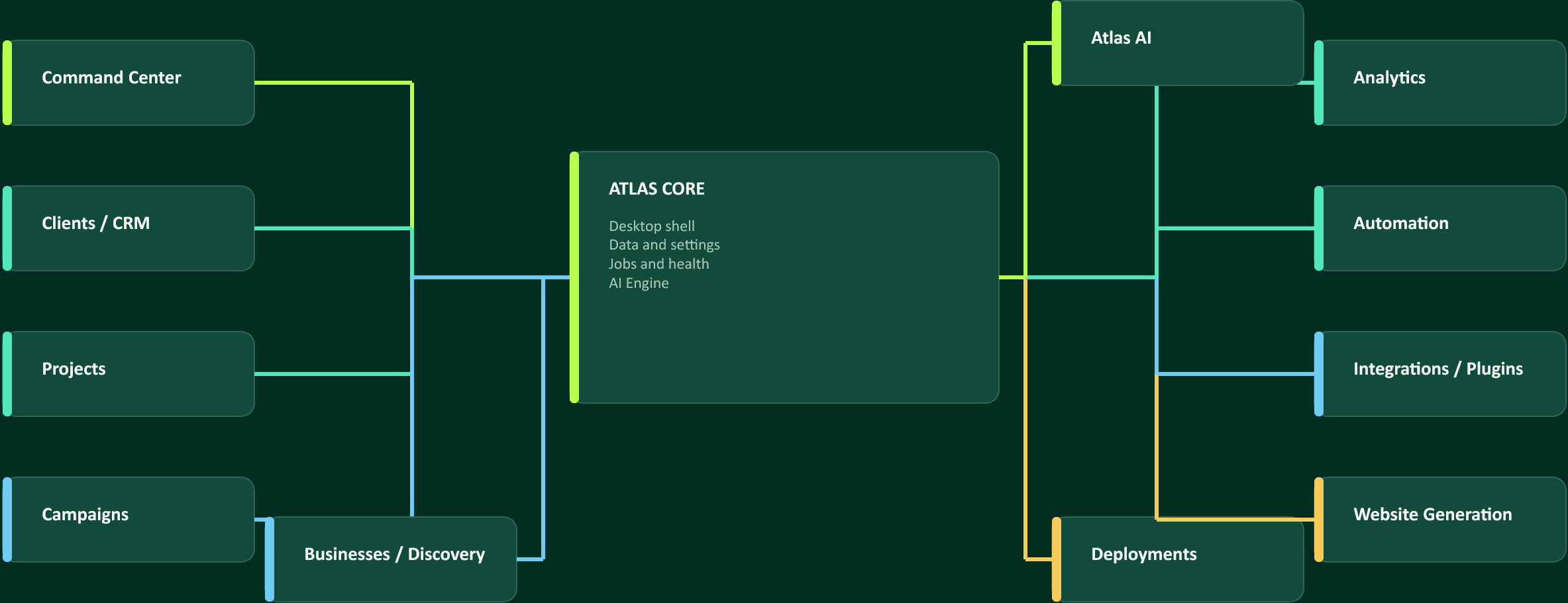
Loopback-only Ollama access, bounded timeouts, normalized errors and output validation constrain AI calls.

06 Operational hygiene

Logging and secret redaction reduce accidental disclosure during diagnostics.

Future cloud providers require explicit secret storage, user consent and service-specific data policies before release.

A shared platform supports eleven product modules



Highlighted current entry points: Command Center and Atlas AI. The other modules represent planned product scope.

Atlas may reduce several lightweight coordination tools as modules mature

The product should earn replacement through connected context and workflow depth. Early releases should avoid claiming complete substitutes for mature specialist platforms.

Project trackers

Reduce duplicate task and status tracking when project records and automation become operational.

Internal notes and prompt libraries

Centralize reusable context, prompts and generated outputs around the work they support.

Manual reporting dashboards

Assemble status and analytics from connected records instead of repeated copy and paste.

Shared spreadsheets

Replace fragile client, campaign and business trackers with structured records and governed updates.

Basic CRM workflows

Cover lightweight relationship and pipeline tracking for small agencies before targeting deeper sales operations.

Standalone AI chat for routine work

Use Atlas AI when the task depends on local operational context and controlled output handling.

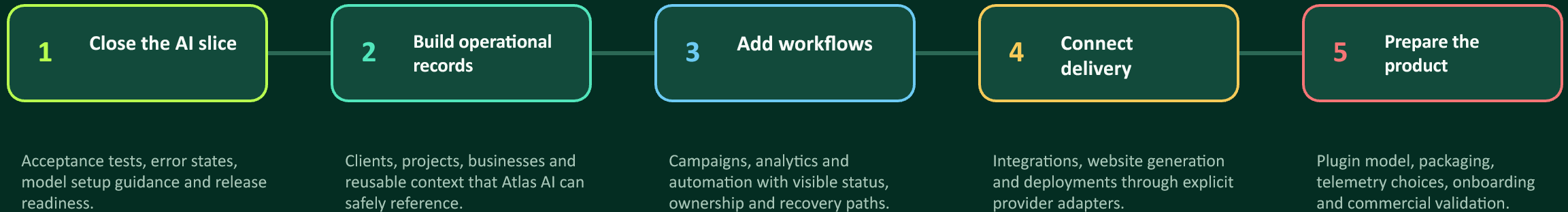
Specialist infrastructure should remain external

CATEGORY	KEEP EXTERNAL	ATLAS ROLE
Communication	Email, calendar and team messaging	Connect activity and approvals to Atlas records.
Finance	Payments, accounting and invoicing	Reference financial events without becoming a ledger.
Delivery infrastructure	Hosting, domains, CDN and deployment platforms	Orchestrate releases through provider APIs.
Marketing platforms	Ad networks, analytics and social channels	Import performance data and trigger governed workflows.
Creative systems	Design, content and media tools	Link assets and handoffs while preserving specialist editors.
Engineering systems	Source control, CI and issue tracking	Connect product delivery to established developer workflows.

Illustrative provider examples can be chosen during implementation. This document does not declare completed integrations.

Five stages move Atlas from a working AI slice to an operating system

Sequence is directional. Timing and scope remain product decisions.



Stage gate for every release

Verify user value, data boundaries, failure handling and maintainability before widening automation or external access.

Commercial direction should follow repeatable operator value

Initial customer

Small agencies and multi-business operators who feel the cost of tool sprawl and repeated context transfer.

Packaging hypothesis

A paid desktop product with capability tiers for modules, collaboration, automation and managed integrations.

Revenue hypothesis

Subscription or license revenue, with cloud AI usage handled transparently when optional providers are enabled.

BRAND DIRECTION

PROJECT ATLAS remains a working codename

Choose the final name after the product promise, target customer and trademark landscape are clear. The brand should convey trusted control, calm operations and capable assistance.

Near-term proof: close Task 007, observe repeated use, then validate which operational record should become the next module.